Notice technique

introduction

Étant donné que nous sommes un groupe de septs étudiants nous avons choisi de faire une première implémentation avec un algorithme relativement simple puis d'essayer une version plus optimisée. En effet, une partie de notre groupe à travaillé sur un algorithme naïf en force brute et le reste sur celui de Fortune. Dans un premier temps, nous avons fait un travail de recherche et de mise en commun de notre compréhension du concept du diagramme de voronoï [1]. Ensuite, nous avons commencé nos implémentations sans intelligence artificielle. Enfin, après avoir rencontré de nombreux problèmes, nous avons obtenu des résultats satisfaisants mais perfectibles, et nous avons hâte de passer à la phase deux.

choix techniques

Pour nos applications respectives nous avons choisi de coder en python car c'est un langage efficace et que nous maîtrisons bien. Deux autres cours du semestre six sont en python et les bibliothèques matplotlib numpy sont pratiques pour ce genre de projet de visualisation mathématique. Enfin, la possibilité de faire de l'orienté objet simplement et de réinvestir nos compétences acquises en R6 A05 et R6 A08 a entériné notre choix.

méthode de construction du diagramme de Voronoï

Avec trois implémentations différentes nous avons trois constructions différentes. La première est la plus intuitive, dans l'algorithme de force brute il faut tester pour chaque pixel quel est son site le plus proche. Notre première méthode consiste donc à diviser un cadre en petites portions rectangulaires et pour chacune d'entre elles calculer la distance euclidienne entre le centre de ces "cases" et tous les sites. Puis trouver la plus petite distance et colorier la case de la même couleur que le site correspondant. La complexité temporelle de cette algorithme n'est pas optimale (O(n log n * N) avec n le nombre de sites donc n log n pour le calcul de distance et le tri et N le nombre de cases donc dans l'idéal le nombre de pixel de notre image finale) mais la complexité cognitive est faible car le brut-force est intuitif.
Pour les deux autres algorithmes, nous avons essayé une autre approche impliquant des intersections de paraboles, une "beach-line" et une ligne de balayage. Par définition, les paraboles sont l'ensemble des points équidistants d'un foyer F et d'une droite directrice d, par conséquent les intersections entre deux paraboles ayant pour foyer des sites sont les

---

[1] The Beauty of Voronoi Diagrams ,
Voronoi Diagram (1/5) | Computational Geometry - Lecture 07 ,
https://fr.wikipedia.org/wiki/Diagramme_de_Voronoï

points des segments d'un diagramme de voronoï. Il faut donc balayer dans la même direction le plan contenant les sites pour trouver les points de segments et les sommets (intersections de trois paraboles) du diagramme de voronoï. Pour cela l'implémentation de Ioan déplace pas à pas une ligne de balayage et calcule les intersections pour trouver les points des segments ce qui s'approche de l'algorithme optimisé de Fortune. Enfin, nous voulions essayer d'implémenter une version complète et optimale de l'algorithme de fortune c'est pourquoi à ce stade nous nous sommes autorisé à nous appuyer sur une structure de départ proposée par le wiki fortune_algorithme[2]. Les classes des objets de base, la formule pour l'intersection de deux paraboles, et la structure initiale de l'algorithme étant bien documentée, nous avons pu construire une classe Beach et le fichier main pour un résultat satisfaisant. Dans cette dernière version, la ligne de balayage se déplace par saut en fonction des événements dans la file d'attente jusqu'à ce qu'elle soit vide. La subtilité de cet algorithme est la détection des segments à "refermer" et des points à enlever de la "beach line" en fonction de si l'événement dans la file est un cercle. Si trois points consécutifs dans la plage ("beach line") forment un cercle, le centre de ce cercle est un sommet du diagramme de voronoï et la ligne de balayage continue d'avancer.

Architecture

Chaque projet contient des classes pour les objets de base comme les points, les cercles, les segments etc… un service qui effectue les calculs et un fichier main responsable de l'affichage. Les données sont dans un fichier txt ou json et un requirement.txt permet une installation rapide sur un environnement virtuel python.

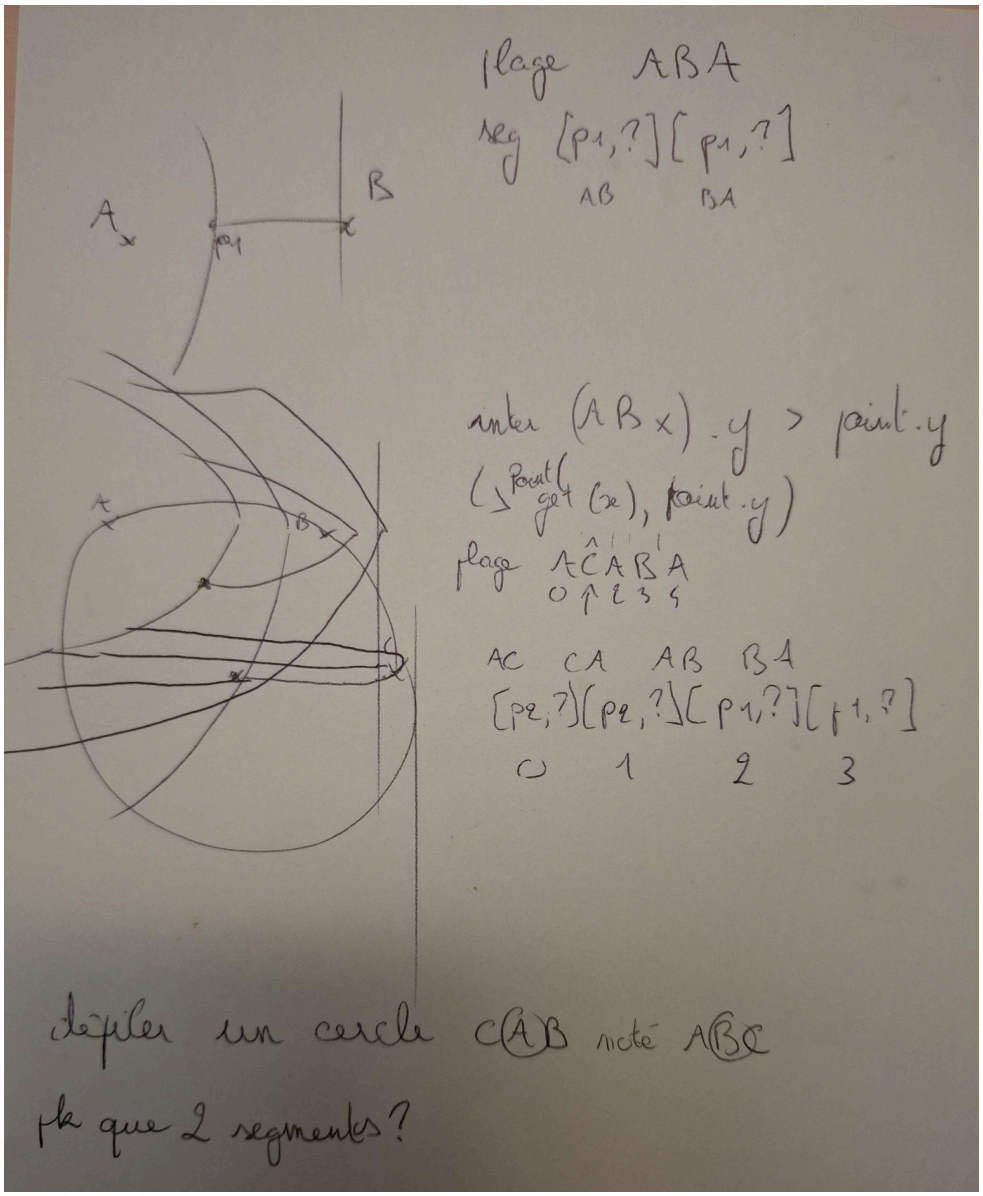
Limites

Que ce soit les performances pour un grand nombre de points ou l'exactitude du diagramme obtenu, nos implémentations ont des limites, mais l'IA saura-t-elle les repousser dans la phase deux ?

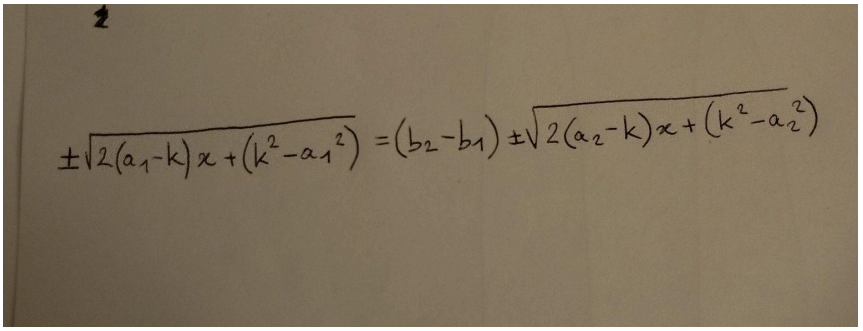
[2] wiki.goupill.fr/doku.php?id=nsi:tds:carte:fortune_algorithme#dokuwiki__topa

Annexes :

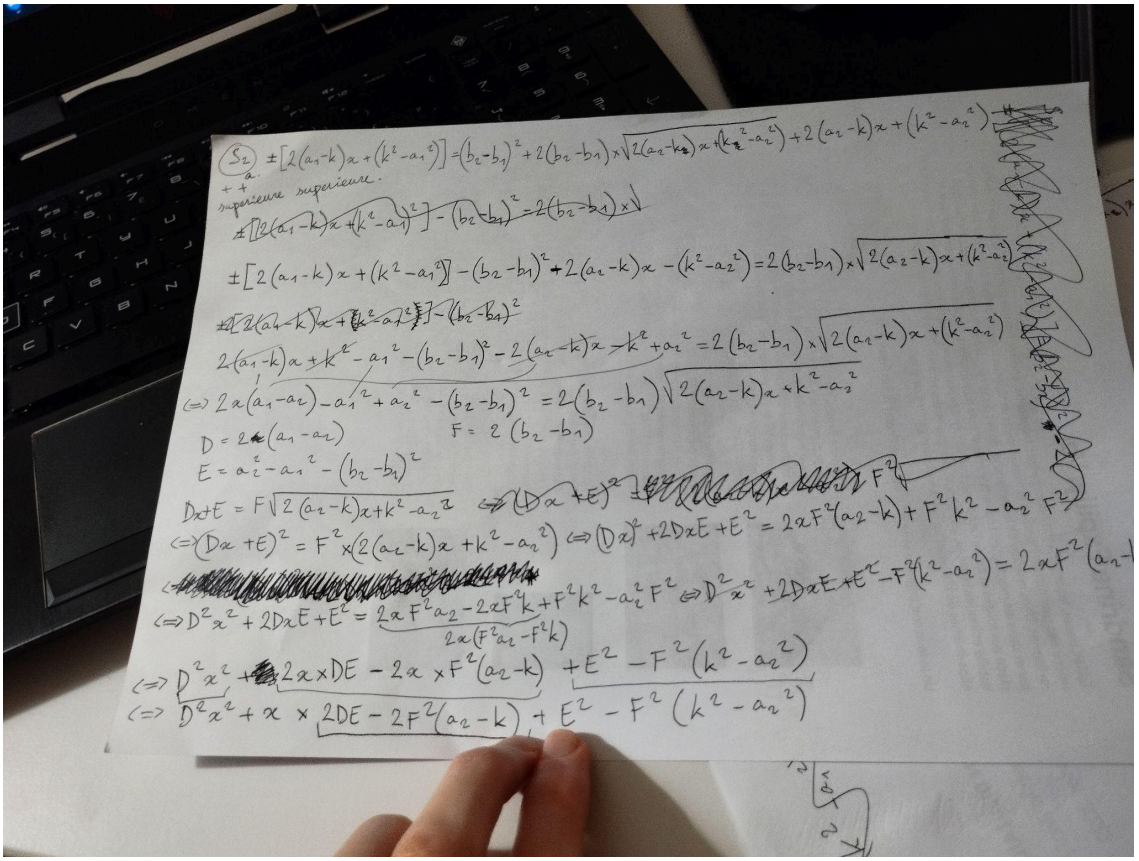
preuve de vie 1, insertion d'un point dans la plage et event cercle



preuve de vie 2, calcule d'intersection entre deux paraboles



$$\pm\sqrt{2(a_1-k)x+(k^2-a_1^2)} = (b_2-b_1) \pm \sqrt{2(a_2-k)x+(k^2-a_2^2)}$$



(S2) $\pm[2(a_1-k)x+(k^2-a_1^2)] = (b_2-b_1)^2 + 2(b_2-b_1)\times\sqrt{2(a_2-k)x+(k^2-a_2^2)} + 2(a_2-k)x+(k^2-a_2^2)$

a.

++

supérieure supérieure.

$\cancel{\pm[2(a_1-k)x+(k^2-a_1^2)] - (b_2-b_1)^2 = 2(b_2-b_1)\times}$

$\pm[2(a_1-k)x+(k^2-a_1^2)] - (b_2-b_1)^2 + 2(a_2-k)x - (k^2-a_2^2) = 2(b_2-b_1)\times\sqrt{2(a_2-k)x+(k^2-a_2^2)}$

$\cancel{\pm[2(a_1-k)x+(k^2-a_1^2)] - (b_2-b_1)^2}$

$2(a_1-k)x \pm k^2 - a_1^2 - (b_2-b_1)^2 - 2(a_2-k)x - k^2 + a_2^2 = 2(b_2-b_1)\times\sqrt{2(a_2-k)x+(k^2-a_2^2)}$

$\Leftrightarrow 2x(a_1-a_2) - a_1^2 + a_2^2 - (b_2-b_1)^2 = 2(b_2-b_1)\sqrt{2(a_2-k)x+k^2-a_2^2}$

$D = 2(a_1-a_2)$ $\qquad F = 2(b_2-b_1)$

$E = a_2^2 - a_1^2 - (b_2-b_1)^2$

$Dx+E = F\sqrt{2(a_2-k)x+k^2-a_2^2} \Leftrightarrow (Dx+E)^2 = F^2$

$\Leftrightarrow (Dx+E)^2 = F^2\times(2(a_2-k)x+k^2-a_2^2) \Leftrightarrow (Dx)^2 + 2DxE + E^2 = 2xF^2(a_2-k) + F^2k^2 - a_2^2F^2$

$\Leftrightarrow D^2x^2 + 2DxE + E^2 = \underbrace{2xF^2a_2 - 2xF^2k}_{2x(F^2a_2-F^2k)} + F^2k^2 - a_2^2F^2 \Leftrightarrow D^2x^2 + 2DxE + E^2 - F^2(k^2-a_2^2) = 2xF^2(a_2-$

$\Leftrightarrow D^2x^2 + 2x\times DE - 2x\times F^2(a_2-k) + E^2 - F^2(k^2-a_2^2)$

$\Leftrightarrow D^2x^2 + x\times\underbrace{2DE - 2F^2(a_2-k)} + E^2 - F^2(k^2-a_2^2)$